Especificaciones Técnicas: Patrones Command e Interpreter

Oct 26, 2026 · @Francisco

Propósito y alcance

Este documento traspasa los proyectos Command e Interpreter a los nuevos desarrolladores: describe cómo están hoy, qué ampliación se les va a agregar y cómo quedan después.

Cada proyecto sigue el mismo orden:

1. Estado actual, con su diagrama de clases.
2. Especificación de la ampliación, con nombres de clases, atributos, métodos y SQL.
3. Diagrama de clases del proyecto con la ampliación ya aplicada.

Los dos proyectos son independientes. Usan Java con Swing para la interfaz y MySQL por JDBC para la persistencia. Las ampliaciones son pequeñas y se pueden asignar a personas distintas sin que se pisen entre sí.

En los diagramas, las clases marcadas como nuevas son las que se crean en la ampliación y las marcadas como modificadas son las que ya existen y cambian.

Entorno y configuración común

Ambos proyectos se ejecutan desde IntelliJ IDEA y comparten la misma base de datos MySQL, `patrones`, con tablas distintas para cada uno.

Elemento	Valor
Lenguaje	Java 8 o superior (lambdas y try-with-resources)
Interfaz gráfica	Swing (`JFrame` , `JTable` , `JComboBox` , `JOptionPane`)
Base de datos	MySQL 8, servidor en `localhost:3307` , base `patrones`

Elemento	Valor
Driver	MySQL Connector/J, agregado en Project Structure > Modules > Dependencies con scope Compile
Acceso a datos	JDBC con `PreparedStatement` y try-with-resources, a través de clases DAO
Paquetes actuales	`Command` e `Interpreter`, con todas las clases en un solo paquete cada uno

Conexión

Cada proyecto tiene su propia clase `ConexionBD` con tres constantes: `URL`, `USER` y `PASS`. La URL es `jdbc:mysql://localhost:3307/patrones?serverTimezone=UTC`. No debe llevar `useSSL=false`: con MySQL 8 provoca el error `Public Key Retrieval is not allowed`. La contraseña de `PASS` es un valor de ejemplo y cada desarrollador pone la suya.

`getConexion()` carga el driver con `Class.forName("com.mysql.cj.jdbc.Driver")`. Si el JAR no está en el classpath del módulo que ejecuta el programa, lanza una `SQLException` con un mensaje claro en lugar de `No suitable driver`.

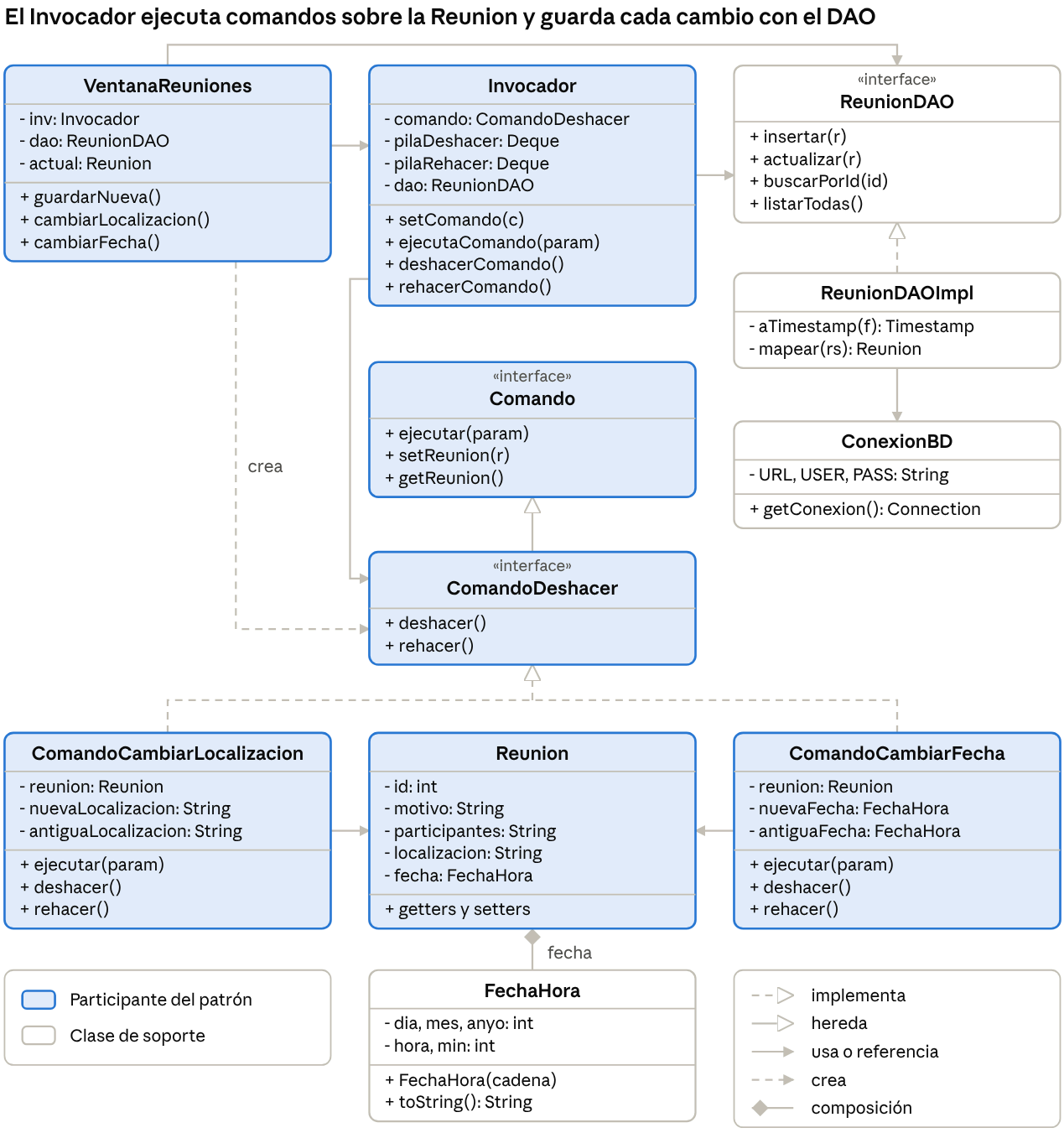
Convenciones del código

- Nombres de clases, métodos y variables en español, en `camelCase`, como el resto del repositorio.
- Toda la lógica de acceso a datos vive en una interfaz DAO y su implementación `...DAOImpl`; la interfaz gráfica nunca escribe SQL.
- Las clases de negocio no imprimen en consola ni abren diálogos. Devuelven valores o lanzan excepciones y la ventana decide cómo mostrarlos.
- Todo error llega a la ventana y se muestra con `JOptionPane`. No se usa `System.err` para errores que el usuario debe ver.
- Cada proyecto incluye su `schema.sql`. Cualquier tabla nueva se agrega a ese script con `CREATE TABLE IF NOT EXISTS`.
- Cada proyecto tiene una guía didáctica en Markdown (`Guia_Didactica_Command.md`, `Guia_Didactica_Interpreter.md`), sin emojis. Si cambian las clases del patrón, hay que actualizar su guía.

Command: estado actual

La aplicación edita una `Reunion` (motivo, participantes, localización y fecha) desde una ventana Swing, con deshacer y rehacer. La ventana crea un comando, se lo entrega al `Invocador` y este lo ejecuta, lo apila y guarda la reunión en MySQL a través de `ReunionDAO`. Hoy existen dos comandos: cambiar localización y cambiar fecha.

Diagrama de clases actual



clases de Command hoy · 11 clases, 2 comandos

La ventana crea el comando y se lo entrega al Invocador. El Invocador lo ejecuta, lo apila para deshacer y rehacer, y guarda la Reunion con el DAO.

Clases y responsabilidades

Clase	Rol	Responsabilidad
`Comando`	Command (interfaz)	Declara `ejecutar(String param)`, `setReunion` y `getReunion`
`ComandoDeshacer`	Command con historial	Extiende `Comando` y agrega `deshacer()` y `rehacer()`
`ComandoCambiarLocalizacion`	ConcreteCommand	Guarda `antiguaLocalizacion` y `nuevaLocalizacion` y las aplica sobre la `Reunion`
`ComandoCambiarFecha`	ConcreteCommand	Guarda `antiguaFecha` y `nuevaFecha` (`FechaHora`) y las aplica sobre la `Reunion`
`Reunion`	Receiver	Datos de la reunión: `id`, `motivo`, `participantes`, `localizacion`, `fecha`
`FechaHora`	Valor	`anyo`, `mes`, `dia`, `hora`, `min`. Se crea desde un texto `d/m/aaaa/h:mm`
`Invocador`	Invoker	Ejecuta, deshace y rehace comandos con dos pilas, y llama al DAO después de cada operación
`ReunionDAO`, `ReunionDAOImpl`	Acceso a datos	`insertar`, `actualizar`, `buscarPorId`, `listarTodas`
`ConexionBD`	Utilidad	`getConexion()` abre la conexión JDBC
`VentanaReuniones`	Client y UI	Formulario, tabla de reuniones y botones; crea los comandos y los pasa al `Invocador`
`Cliente`	Punto de entrada	`main` que abre la ventana

Base de datos actual

Una sola tabla, `reunion`.

Columna	Tipo	Notas
`id`	`INT AUTO_INCREMENT`	Clave primaria
`motivo`	`VARCHAR(100)`	No nulo
`participantes`	`VARCHAR(255)`	Lista en texto libre
`localizacion`	`VARCHAR(100)`	
`fecha`	`DATETIME`	No nulo; se arma desde `FechaHora`

Deuda técnica conocida

- Cada ejecución necesita un objeto comando nuevo. Si se reutiliza uno, sobrescribe su valor antiguo y el deshacer falla.
- Las pilas de deshacer y rehacer viven en memoria y se pierden al cerrar la ventana.
- El `Invocador` cambia la `Reunion` en memoria y después llama al DAO. Si el DAO falla, la memoria y la base quedan distintas.
- `FechaHora(String)` lanza una excepción sin controlar si el texto no tiene el formato `d/m/aaaa/h:mm`, y no valida rangos (por ejemplo, mes 13).
- No se valida que la localización no esté vacía.
- La ventana guarda las reuniones en un `Map` de sesión (`cache`). No refleja cambios hechos por otro usuario en la base.

Command: especificación de la ampliación

La ampliación agrega dos comandos nuevos, cambiar motivo y cambiar participantes, y un historial persistente que registra en la base cada ejecución, deshacer y rehacer. La ventana muestra ese historial en una tabla. Son 6 clases nuevas, 5 clases modificadas y 1 tabla nueva.

Requisitos funcionales

1. El usuario puede cambiar el motivo y los participantes de la reunión seleccionada, y deshacer y rehacer esos cambios igual que los de localización y fecha.

2. Cada vez que el `Invocador` ejecuta, deshace o rehace un comando, se guarda un registro en la tabla `historial_comando`.
3. Al seleccionar una reunión, la ventana muestra su historial, del más reciente al más antiguo.
4. Los campos de texto del formulario no pueden quedar vacíos al ejecutar un comando. Si lo están, la ventana muestra el mensaje de error y no se ejecuta nada.

Clases nuevas

Clase	Atributos	Métodos
`ComandoCambiarMotivo` (implementa `ComandoDeshacer`)	`reunion: Reunion`, `nuevoMotivo: String`, `antiguoMotivo: String`	`ejecutar(String param)`, `deshacer()`, `rehacer()`, `getReunion()`, `setReunion(Reunion)`, `getNombre()`, `getValorAnterior()`, `getValorNuevo()`
`ComandoCambiarParticipantes` (implementa `ComandoDeshacer`)	`reunion: Reunion`, `nuevosParticipantes: String`, `antiguosParticipantes: String`	Los mismos que `ComandoCambiarMotivo`
`AccionHistorial` (enum)	Valores `EJECUTAR`, `DESHACER`, `REHACER`	
`RegistroHistorial`	`id: int`, `reunionId: int`, `comando: String`, `accion: AccionHistorial`, `valorAnterior: String`, `valorNuevo: String`, `fechaRegistro: LocalDateTime`	Constructor con `reunionId`, `comando`, `accion`, `valorAnterior`, `valorNuevo`; getters y setters
`HistorialDAO` (interfaz)		`insertar(RegistroHistorial r)`, `listarPorReunion(int reunionId): List<RegistroHistorial>`; ambos lanzan `SQLException`
`HistorialDAOImpl`		Implementa `HistorialDAO` con JDBC. `listarPorReunion` ordena por `id DESC`

`getNombre()` devuelve el nombre corto del comando: `"CambiarMotivo"` y `"CambiarParticipantes"`. `getValorAnterior()` y `getValorNuevo()` devuelven los textos antiguo y nuevo que el comando guardó al ejecutarse.

Clases modificadas

Clase	Cambio
`ComandoDeshacer`	Agrega `String getNombre()`, `String getValorAnterior()` y `String getValorNuevo()`
`ComandoCambiarLocalizacion`	Implementa los tres métodos nuevos: nombre `"CambiarLocalizacion"`, y los valores `antiguaLocalizacion` y `nuevaLocalizacion`
`ComandoCambiarFecha`	Implementa los tres métodos nuevos: nombre `"CambiarFecha"`, y los valores `antiguaFecha.toString()` y `nuevaFecha.toString()`
`Invocador`	Agrega el atributo `historialDao: HistorialDAO`. El constructor pasa a ser `Invocador(ReunionDAO dao, HistorialDAO historialDao)`. Agrega el método privado `registrar(ComandoDeshacer c, AccionHistorial accion) throws SQLException`, que se llama al final de `ejecutaComando`, `deshacerComando` y `rehacerComando`, después de `dao.actualizar`
`VentanaReuniones`	Agrega `historialDao`, los botones `btnMotivo` y `btnParticipantes`, la tabla `tblHistorial` con su `modeloHistorial`, y los métodos `cambiarMotivo()`, `cambiarParticipantes()` y `refrescarHistorial()`

Qué guarda `registrar`

Acción	`valorAnterior`	`valorNuevo`
`EJECUTAR`	`c.getValorAnterior()`	`c.getValorNuevo()`

Acción	`valorAnterior`	`valorNuevo`
`DESHACER`	`c.getValorNuevo()`	`c.getValorAnterior()`
`REHACER`	`c.getValorAnterior()`	`c.getValorNuevo()`

El campo `comando` guarda `c.getNombre()` y `reunionId` guarda `c.getReunion().getId()`. En la base, `accion` guarda `accion.name()`.

Cambios en la ventana

- Los botones del panel inferior pasan a ser: Guardar nueva, Cambiar motivo, Cambiar participantes, Cambiar localización, Cambiar fecha, Deshacer y Rehacer.
- El centro de la ventana pasa a ser un `JSplitPane` vertical: la tabla de reuniones arriba y la tabla de historial abajo, con las columnas Fecha, Comando, Acción, Valor anterior y Valor nuevo.
- `cambiarMotivo()` y `cambiarParticipantes()` siguen el mismo esquema que `cambiarLocalizacion()`: verifican la selección, crean el comando, llaman a `setReunion`, a `inv.setComando` y a `inv.ejecutaComando` con el texto del campo.
- `refrescarHistorial()` consulta `historialDao.listarPorReunion(actual.getId())` y rellena `modeloHistorial`. Se llama desde `accion(...)`, después de `refrescarTabla()`, y también al cambiar la selección de la tabla de reuniones.
- Antes de ejecutar un comando, `cambiarMotivo()`, `cambiarParticipantes()` y `cambiarLocalizacion()` lanzan `IllegalArgumentException` con un mensaje claro si el campo de texto está vacío (por ejemplo `"Escribe el nuevo motivo."`).

Base de datos

Se agrega este bloque a `schema.sql`.

```
CREATE TABLE IF NOT EXISTS historial_comando (
    id             INT AUTO_INCREMENT PRIMARY KEY,
    reunion_id     INT NOT NULL,
    comando        VARCHAR(50) NOT NULL,
    accion         VARCHAR(10) NOT NULL,
    valor_anterior VARCHAR(255),
    valor_nuevo    VARCHAR(255),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (reunion_id) REFERENCES reunion(id)
);
```

Orden de implementación sugerido

1. Agregar la tabla a `schema.sql` y ejecutarla.
2. Crear `AccionHistorial`, `RegistroHistorial`, `HistorialDAO` y `HistorialDAOImpl`.
3. Ampliar `ComandoDeshacer` e implementar los tres métodos nuevos en los dos comandos existentes.
4. Crear `ComandoCambiarMotivo` y `ComandoCambiarParticipantes`.
5. Modificar `Invocador` (atributo, constructor y `registrar`).
6. Modificar `VentanaReuniones` (botones, tabla de historial, validaciones y `new Invocador(dao, historialDao)`).
7. Actualizar `Guia_Didactica_Command.md` con las clases nuevas.

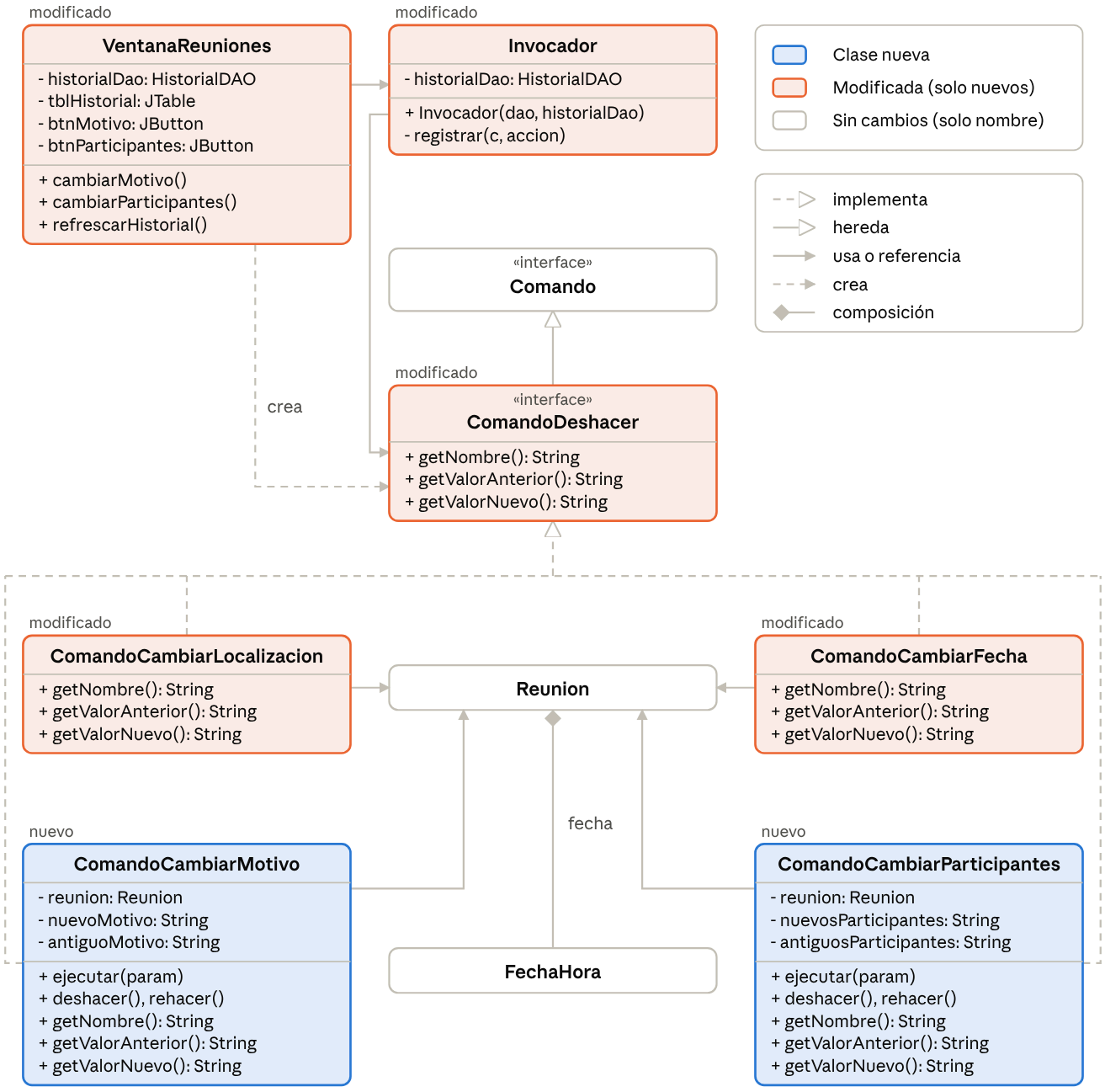
Fuera de alcance

No incluye eliminar reuniones, restaurar las pilas de deshacer desde el historial al abrir la aplicación, ni transacciones que unan la actualización de la reunión con el registro del historial.

Command: diagrama de clases con la ampliación

Son dos diagramas: el núcleo del patrón y la persistencia del historial. Cada clase lleva la etiqueta `nuevo` o `modificado`, con un relleno distinto, y las modificadas muestran solo sus miembros nuevos. Las clases sin cambios aparecen solo con su nombre; sus miembros están en el diagrama del estado actual.

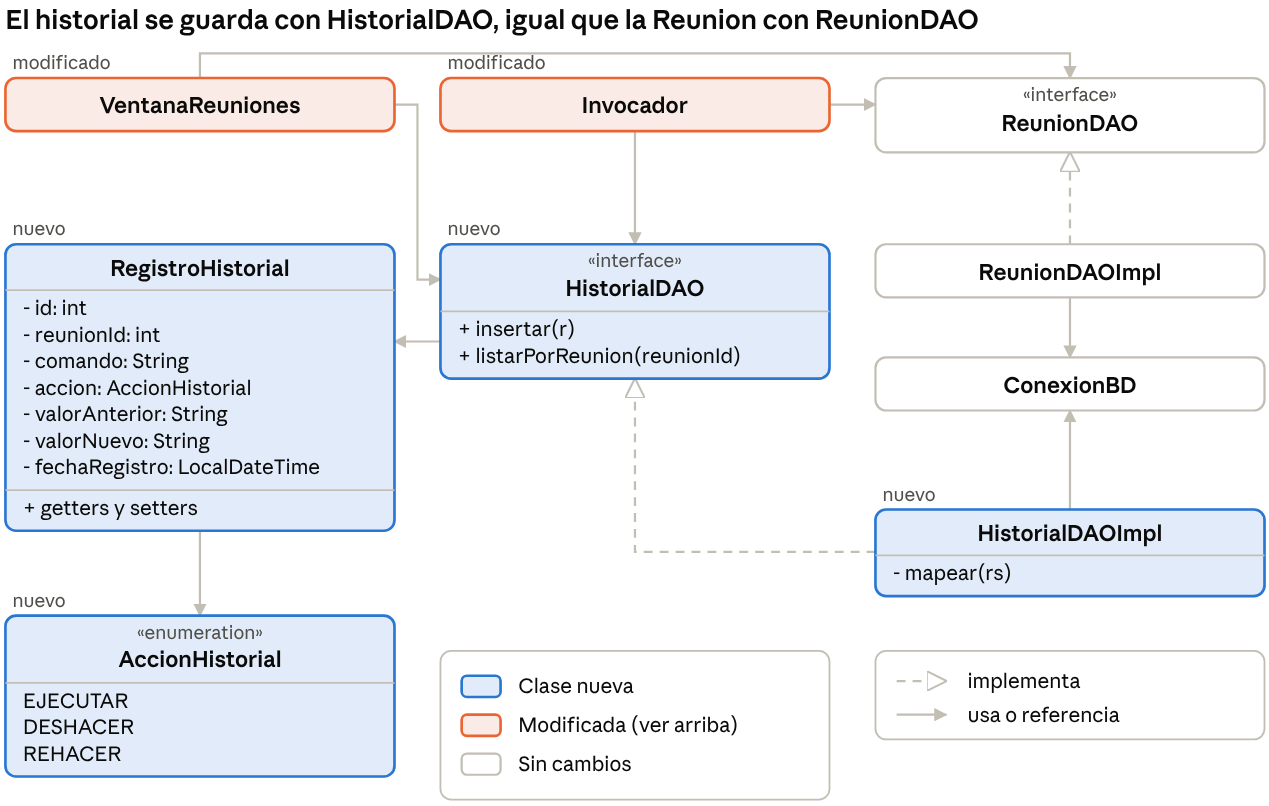
Dos comandos nuevos se suman al patrón y el Invocador registra cada operación



núcleo del patrón con la ampliación · 2 comandos nuevos, 5 clases modificadas

El Invocador sigue ejecutando cualquier ComandoDeshacer. Los dos comandos nuevos entran por la misma interfaz y los tres métodos nuevos de ComandoDeshacer le dan al Invocador lo que necesita para registrar cada operación.

Persistencia e historial



persistencia e historial · 4 clases nuevas, Invocador y ventana como origen

El Invocador y la ventana usan HistorialDAO igual que ya usan ReunionDAO. HistorialDAO trabaja con RegistroHistorial, que guarda la acción como un valor de AccionHistorial.

Interpreter: estado actual

La aplicación responde consultas escritas en texto sobre empresas y empleados, por ejemplo `muestra empleados por empresas`. La ventana lee los datos de MySQL y los carga en un `Contexto`. Después, el `InterpreteCliente` analiza la frase, elige el intérprete que corresponde y devuelve el resultado como texto, que la ventana muestra en pantalla.

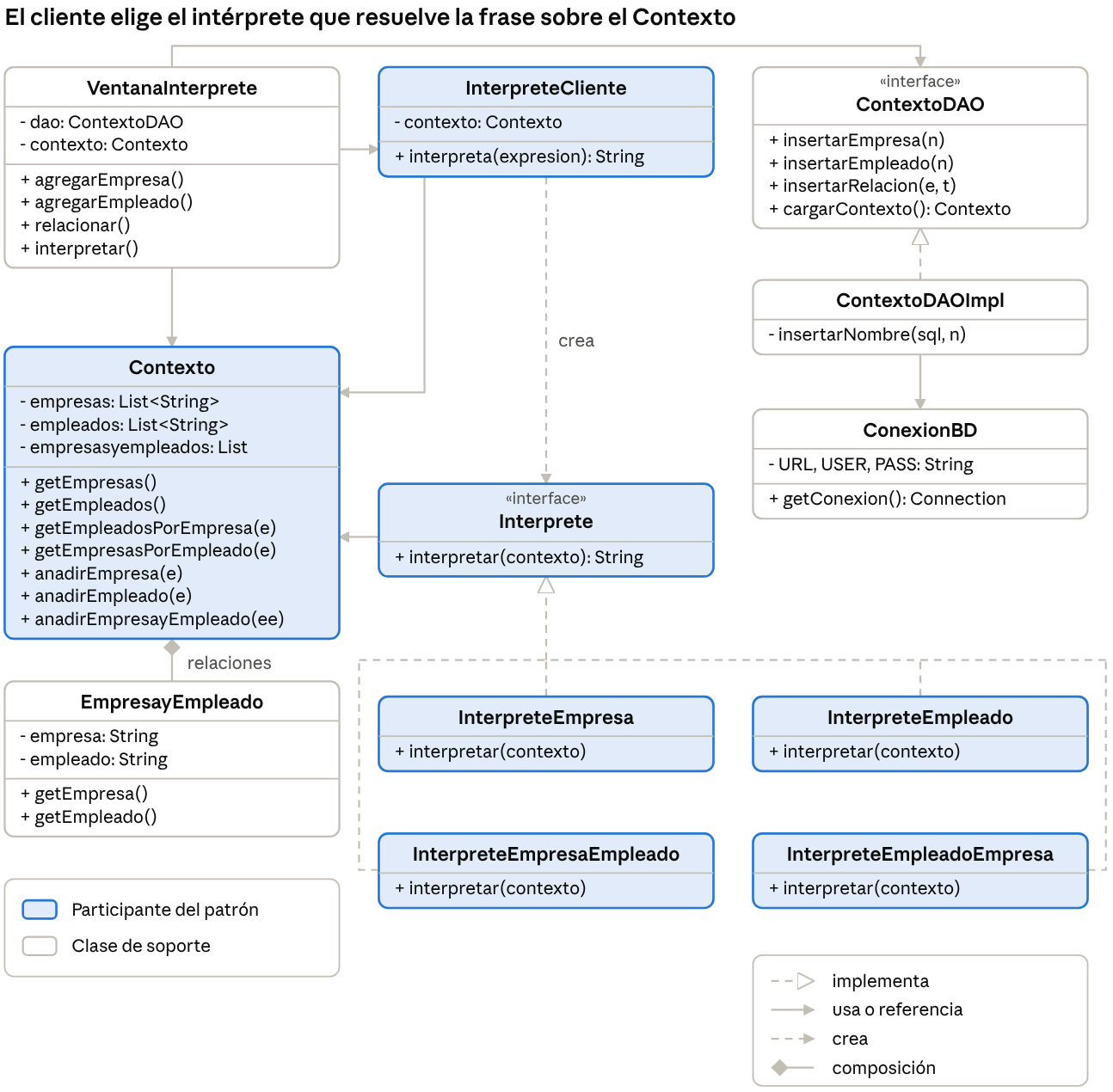
Gramática actual

```
consulta ::= "muestra" objeto [ "por" objeto ]
objeto   ::= "empresas" | "empleados"
```

Expresión	Intérprete	Resultado
`muestra empresas`	`InterpreteEmpresa`	Lista de empresas
`muestra empleados`	`InterpreteEmpleado`	Lista de empleados
`muestra empleados por empresas`	`InterpreteEmpleadoEmpresa`	Cada empresa con sus empleados
`muestra empresas por empleados`	`InterpreteEmpresaEmpleado`	Cada empleado con sus empresas

Los nombres de `InterpreteEmpleadoEmpresa` e `InterpreteEmpresaEmpleado` quedaron al revés de lo que muestran. Se mantienen así para no romper el código existente.

Diagrama de clases actual



clases de Interpreter hoy · 12 clases, 4 intérpretes

La ventana carga el Contexto desde la base. El InterpreteCliente analiza la frase, crea el intérprete que corresponde y le pasa el Contexto para obtener el texto del resultado.

Clases y responsabilidades

Clase	Rol	Responsabilidad
`Interprete`	AbstractExpression (interfaz)	Declara `String interpretar(Contexto contexto)`
`InterpreteEmpresa`, `InterpreteEmpleado`	TerminalExpression	Devuelven la lista de empresas o de empleados
`InterpreteEmpleadoEmpresa`, `InterpreteEmpresaEmpleado`	TerminalExpression	Recorren una lista y consultan al `Contexto` la lista relacionada de cada elemento
`Contexto`	Context	Guarda `empresas`, `empleados` y las relaciones; ofrece `getEmpleadosPorEmpresa` y `getEmpresasPorEmpleado`
`EmpresayEmpleado`	Valor	Relación inmutable entre una `empresa` y un `empleado`
`InterpreteCliente`	Client	`interpreta(String expresion)` recorre las palabras de la frase, decide el intérprete y lo ejecuta
`ContextoDAO`, `ContextoDAOImpl`	Acceso a datos	`insertarEmpresa`, `insertarEmpleado`, `insertarRelacion`, `cargarContexto`
`ConexionBD`	Utilidad	`getConexion()` abre la conexión JDBC
`VentanaInterprete`	UI	Formulario para agregar datos, campo de expresión y área de resultado
`EjecutarPatron`	Punto de entrada	`main` que abre la ventana

Base de datos actual

Tabla	Columnas	Notas
`empresa`	`id`, `nombre`	`nombre` es único
`empleado`	`id`, `nombre`	`nombre` es único
`empresa_empleado`	`empresa_id`, `empleado_id`	Clave primaria compuesta; claves foráneas a las dos tablas anteriores

`schema.sql` también inserta los datos de ejemplo: IBM, Indra y ONO; Juan, Marta y Javier.

Deuda técnica conocida

- `InterpreteCliente` decide el intérprete con banderas (`mostrar`, `mostrarPor`, `usaPor`) y un `switch`. Cada frase nueva obliga a modificar esa clase. No hay árbol de expresiones.
- Una frase inválida devuelve un texto de ayuda como si fuera un resultado. La ventana no puede distinguir un error de una respuesta.
- El analizador pasa la frase a minúsculas, así que hoy no admite nombres propios dentro de la frase.
- Cada acción de la ventana vuelve a cargar todo el contexto desde la base (tres consultas).
- Si se agrega un nombre repetido, la restricción `UNIQUE` de MySQL lanza un error que se muestra sin traducir.

Interpreter: especificación de la ampliación

La ampliación agrega el verbo `cuenta`, el filtro `de <nombre>`, una excepción propia para las frases inválidas y un historial de consultas guardado en la base. Son 9 clases nuevas, 3 clases modificadas y 1 tabla nueva.

Requisitos funcionales

1. El usuario puede contar empresas y empleados con el verbo `cuenta`.
2. El usuario puede filtrar con `de`: los empleados de una empresa y las empresas de un empleado.

3. Las frases inválidas lanzan `ExpresionInvalidaException`. La ventana muestra el mensaje en el área de resultado.
4. Cada consulta, válida o no, se guarda en `consulta_historial`. La ventana ofrece las últimas 10 en una lista desplegable; al elegir una, se copia al campo de expresión.
5. Los nombres en el filtro `de` no distinguen mayúsculas de minúsculas: `muestra empleados de indra` equivale a `muestra empleados de Indra`.

Gramática ampliada

```
consulta ::= "muestra" objeto [ "por" objeto | "de" nombre ]
           | "cuenta" objeto [ "por" objeto ]
objeto   ::= "empresas" | "empleados"
nombre   ::= una sola palabra, sin espacios
```

Expresión	Intérprete	Resultado de ejemplo
`cuenta empresas`	`InterpreteContarEmpresas`	`Total de empresas: 3`
`cuenta empleados`	`InterpreteContarEmpleados`	`Total de empleados: 3`
`cuenta empleados por empresas`	`InterpreteContarPorEmpresa`	Una línea por empresa: `Indra: 2`
`muestra empleados de Indra`	`InterpreteEmpleadosDeEmpresa`	`Empleados de Indra: Juan, Javier`
`muestra empresas de Juan`	`InterpreteEmpresasDeEmpleado`	`Empresas de Juan: Indra, ONO`

Reglas de validación, todas con `ExpresionInvalidaException`:

- La frase debe empezar con `muestra` o `cuenta`.
- `de` solo se acepta con `muestra` y debe ir seguido de un nombre.
- En `muestra empleados de X`, `X` debe ser una empresa existente. En `muestra empresas de X`, `X` debe ser un empleado existente. El mensaje es `No existe la empresa: X` o `No existe el empleado: X`.
- `cuenta` no acepta `de`.
- Si no se reconoce el objeto (`empresas` o `empleados`), el mensaje incluye la lista de ejemplos que hoy devuelve `InterpreteCliente`.

Clases nuevas

Clase	Atributos	Métodos
`InterpreteContarEmpresas` (implementa `Interprete`)		`interpretar(Contexto contexto): String`
`InterpreteContarEmpleados` (implementa `Interprete`)		`interpretar(Contexto contexto): String`
`InterpreteContarPorEmpresa` (implementa `Interprete`)		`interpretar(Contexto contexto): String`
`InterpreteEmpleadosDeEmpresa` (implementa `Interprete`)	`empresa: String`	Constructor `InterpreteEmpleadosDeEmpresa(String empresa)`, `interpretar(Contexto contexto): String`
`InterpreteEmpresasDeEmpleado` (implementa `Interprete`)	`empleado: String`	Constructor `InterpreteEmpresasDeEmpleado(String empleado)`, `interpretar(Contexto contexto): String`
`ExpresionInvalidaException` (extiende `Exception`)		Constructor `ExpresionInvalidaException(String mensaje)`
`RegistroConsulta`	`id: int`, `expresion: String`, `valida: boolean`, `fechaRegistro: LocalDateTime`	Constructor con `expresion` y `valida`; getters y setters
`HistorialConsultaDAO` (interfaz)		`insertar(RegistroConsulta r)`, `listarUltimas(int limite): List<RegistroConsulta>`; ambos lanzan `SQLException`
`HistorialConsultaDAOImpl`		Implementa `HistorialConsultaDAO` con JDBC. `listarUltimas` ordena por `id DESC` y usa `LIMIT ?`

Clases modificadas

Clase	Cambio
`Contexto`	Agrega `Optional<String> buscarEmpresa(String nombre)` y `Optional<String> buscarEmpleado(String nombre)`. Comparan sin distinguir mayúsculas y devuelven el nombre tal como está guardado
`InterpreteCliente`	`interpreta(String expresion)` pasa a lanzar `ExpresionInvalidaException`. Reconoce las palabras `cuenta` y `de`. Conserva las mayúsculas del nombre que sigue a `de` y compara las palabras clave en minúsculas. Agrega las variables `accion` (`'M'` muestra, `'C'` cuenta) y `nombre: String`. Cuando la frase es inválida lanza la excepción en lugar de devolver el texto de ayuda
`VentanaInterprete`	Agrega `historialDao: HistorialConsultaDAO`, `cmbHistorial: JComboBox<String>` y el método `refrescarHistorial()`. `interpretar()` captura `ExpresionInvalidaException`, muestra el mensaje en `txtSalida` y registra la consulta con `valida` en `false`; si todo sale bien, la registra con `true`

`Interprete`, `InterpreteEmpresa`, `InterpreteEmpleado`, `InterpreteEmpresaEmpleado`, `InterpreteEmpleadoEmpresa`, `ContextoDAO` y `ContextoDAOImpl` no cambian.

Cambios en la ventana

- Debajo del campo de expresión aparece la lista `cmbHistorial` con las últimas 10 expresiones. Al seleccionar una, se copia a `txtExpresion`; no se ejecuta sola.
- `refrescarHistorial()` consulta `historialDao.listarUltimas(10)` y se llama al abrir la ventana y después de cada `interpretar()`.
- Una expresión inválida no muestra un diálogo de error: el mensaje va al área de resultado para que se lea junto con los ejemplos.

Base de datos

Se agrega este bloque a `schema.sql`.

```
CREATE TABLE IF NOT EXISTS consulta_historial (
    id             INT AUTO_INCREMENT PRIMARY KEY,
    expresion      VARCHAR(200) NOT NULL,
    valida         BOOLEAN NOT NULL,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Orden de implementación sugerido

1. Agregar la tabla a `schema.sql` y ejecutarla.
2. Crear `ExpresionInvalidaException`.
3. Agregar `buscarEmpresa` y `buscarEmpleado` a `Contexto`.
4. Crear los cinco intérpretes nuevos.
5. Modificar `InterpreteCliente`: nuevos tokens, validaciones y excepción.
6. Crear `RegistroConsulta`, `HistorialConsultaDAO` y `HistorialConsultaDAOImpl`.
7. Modificar `VentanaInterprete`: historial y manejo de la excepción.
8. Actualizar `Guia_Didactica_Interpreter.md` con la gramática ampliada y las clases nuevas.

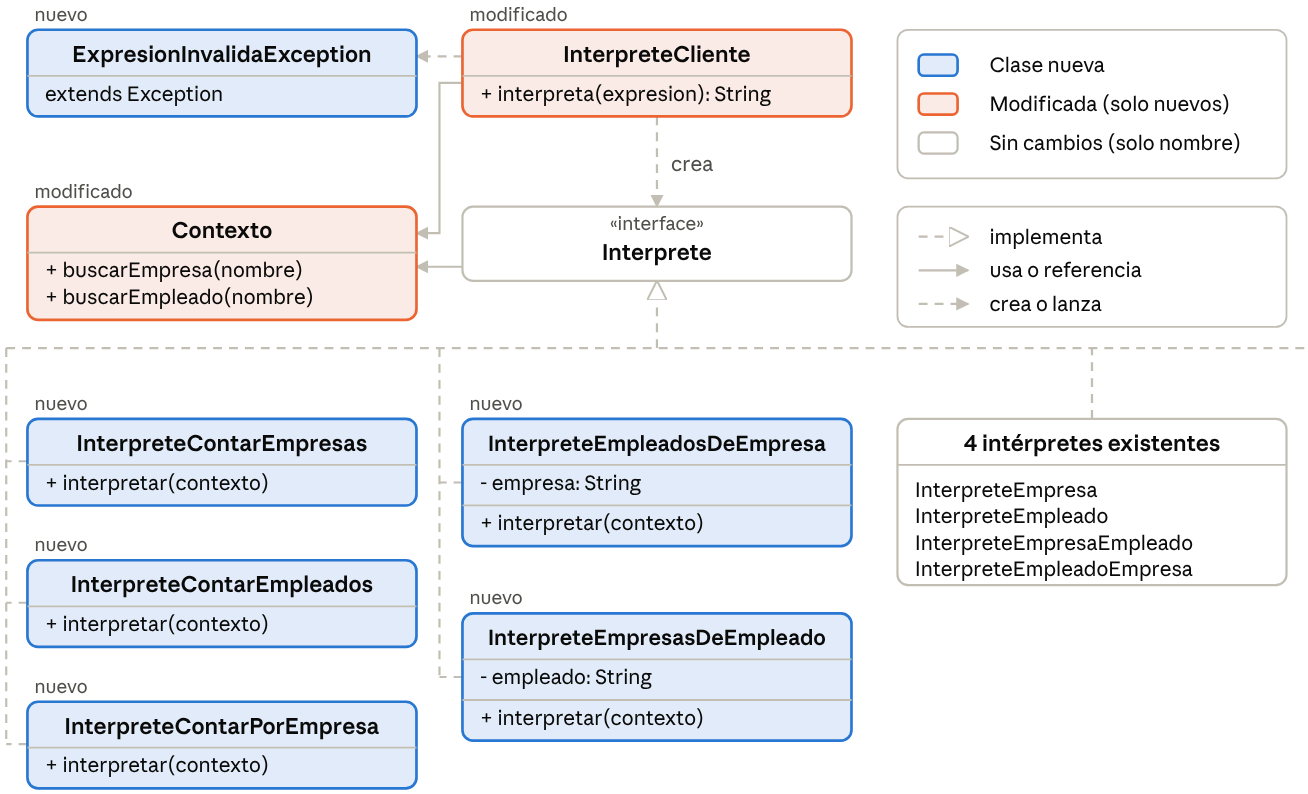
Fuera de alcance

No incluye nombres con espacios, combinar condiciones con `y` u `o`, convertir la frase en un árbol de expresiones ni guardar el contexto en memoria entre consultas.

Interpreter: diagrama de clases con la ampliación

Son dos diagramas: el núcleo del intérprete y el historial de consultas. Cada clase lleva la etiqueta `nuevo` o `modificado`, con un relleno distinto, y las modificadas muestran solo sus miembros nuevos. Las clases sin cambios aparecen solo con su nombre; sus miembros están en el diagrama del estado actual.

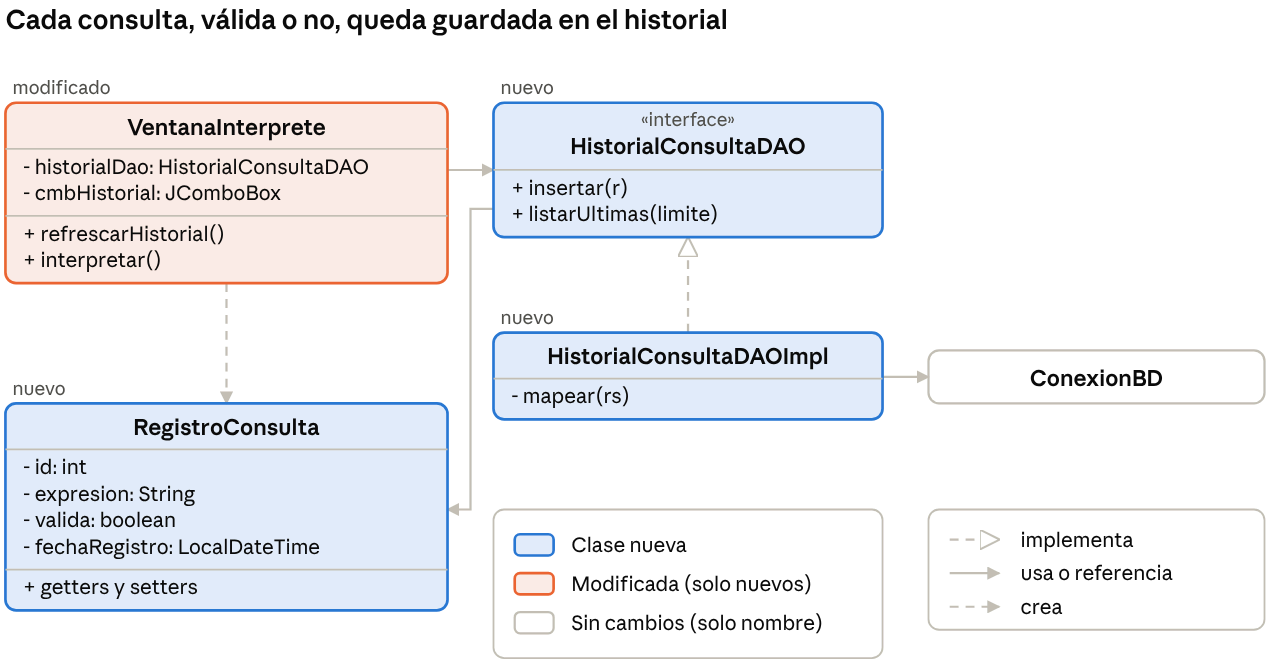
Cinco intérpretes nuevos amplían el lenguaje sin tocar los existentes



núcleo del intérprete con la ampliación · 5 intérpretes y 1 excepción nuevos

Los cinco intérpretes nuevos implementan la misma interfaz Interprete que los cuatro existentes, que no cambian. El InterpreteCliente los crea y lanza ExpresionInvalidaException cuando la frase no es válida.

Historial de consultas



historial de consultas · 3 clases nuevas y la ventana modificada

La ventana guarda cada consulta con HistorialConsultaDAO, que trabaja con RegistroConsulta, y lee de ahí las últimas 10 para la lista desplegable.

Criterios de aceptación y entrega

Cada ampliación se da por terminada cuando pasa todas las pruebas manuales de su tabla y cumple la lista de entrega. Las pruebas de Interpreter suponen los datos de ejemplo de `schema.sql`.

Pruebas manuales de Command

#	Prueba	Resultado esperado
1	Seleccionar una reunión, escribir un motivo nuevo y pulsar Cambiar motivo	La tabla y la base muestran el motivo nuevo. El historial tiene una fila `EJECUTAR` de `CambiarMotivo`
2	Pulsar Deshacer	Vuelve el motivo anterior. El historial agrega una fila `DESHACER` con los valores invertidos

#	Prueba	Resultado esperado
3	Pulsar Rehacer	Vuelve el motivo nuevo. El historial agrega una fila `REHACER`
4	Repetir las pruebas 1 a 3 con Cambiar participantes	Mismo comportamiento, con `CambiarParticipantes` en el historial
5	Pulsar Cambiar motivo, Cambiar participantes o Cambiar localización con el campo vacío	Un diálogo muestra el mensaje de error. No cambia nada y no se agrega ninguna fila al historial
6	Cambiar el motivo, luego la fecha, y pulsar Deshacer dos veces	Se deshace primero la fecha y después el motivo
7	Hacer un cambio, deshacerlo y hacer otro cambio distinto	El botón Rehacer queda deshabilitado
8	Cerrar la aplicación y abrirla otra vez	Las reuniones y el historial siguen en la base. Deshacer y Rehacer están deshabilitados

Pruebas manuales de Interpreter

#	Expresión	Resultado esperado
1	`cuenta empresas`	`Total de empresas: 3`
2	`cuenta empleados`	`Total de empleados: 3`
3	`cuenta empleados por empresas`	`IBM: 1`, `Indra: 2` y `ONO: 1`, cada uno en su línea
4	`muestra empleados de indra`	`Empleados de Indra: Juan, Javier`
5	`muestra empresas de Juan`	`Empresas de Juan: Indra, ONO`
6	`muestra empleados de Acme`	`No existe la empresa: Acme` en el área de resultado, sin diálogo de error
7	`cuenta empresas de Juan`	Mensaje de que `cuenta` no acepta `de`
8	`hola`	Mensaje de expresión inválida con la lista de ejemplos

#	Expresión	Resultado esperado
9	Las cuatro expresiones que ya existían	Mismo resultado que antes de la ampliación
10	Ejecutar las pruebas 1 y 6, y abrir la lista del historial	Aparecen ambas, la más reciente primero. Al elegir una, se copia al campo de expresión

Lista de entrega

- El proyecto compila y se ejecuta desde IntelliJ con el driver MySQL en el classpath del módulo.
- `schema.sql` incluye la tabla nueva y se puede ejecutar varias veces sin error.
- Ninguna clase nueva escribe SQL fuera de un DAO, ni imprime en consola ni abre diálogos fuera de la ventana.
- La guía didáctica del patrón incluye las clases nuevas.
- Si la implementación se aparta de este documento (nombres, firmas o relaciones), se actualizan aquí el texto y el diagrama de clases con la ampliación.